
Trabajo Práctico N° 6

TEMA: Punteros. Mecanismos de pasaje de parámetros. Asignación de memoria dinámica

"Un puntero es una variable que contiene la dirección de una variable". Kernighan & Ritchie

EJERCICIOS SUGERIDOS PARA REPASAR:

I. Calentando motores

1. Escriba un programa que lea dos números enteros y los almacene en las variables a y b. Muestre en pantalla los valores leídos en a y b y sus direcciones de memoria.
2. Cree dos punteros enteros llamados p y q. Asigne en los punteros p y q la dirección de memoria de a y b respectivamente.
3. Muestre el contenido de p y q.
4. Muestre el contenido apuntado por p y q.
5. Muestre las direcciones de memoria de p y q.
6. Lea un número y guárdelo en una variable llamada c. Muestre el contenido de c y la dirección de memoria de la variable c.
7. Asigne a la variable c el valor de a utilizando el puntero p. Muestre las variables a y c.
8. En la variable c guarde la suma de a+b utilizando los punteros p y q. Muestre las variables a, b y c.
9. ¿Qué sucede si ejecutamos *p=*p+1?
10. ¿Qué sucede si ejecutamos p=p+1?
11. Dada la siguiente asignación: p=&a:
12. ¿Las sentencias *p=*p+1 y *p=a+1 son equivalentes?
13. ¿Las sentencias p=p+1 y p=&a+1 son equivalentes?

II. Calentando motores con arreglos

Precalentamiento:

Lea y almacene una frase en minúscula en un arreglo de nombre frase[100]. Cuente las consonantes y conviértelas a mayúsculas.
Use variables punteros para manejar los contadores.

En sus marcas:

Modifique el código de su programa para sumar un puntero que apunte al arreglo frase. Siga las instrucciones paso a paso:

1. Declare en forma local a main una variable puntero **punt**, que apunte a un objeto de tipo char.
2. Asigne la dirección inicial del arreglo frase a la variable puntero **punt**. Realice esta tarea de dos maneras distintas.
3. Muestre en pantalla la dirección de la variable puntero **punt**, use el indicador de formato apropiado. Muestre, mediante el puntero, la dirección de la primera componente del arreglo **frase** y su contenido.

4. Muestre en pantalla el contenido de la cuarta celda del arreglo **frase**. Refiérase a ella utilizando: notación de subíndice del arreglo y notación de puntero con desplazamiento mediante **punt**.
5. Suponiendo que **punt** apunta al principio del arreglo **frase**, ¿cuál es la dirección referenciada por punt+5? ¿Cuál es el valor almacenado en esa dirección?. Al mostrar el valor almacenado en punt+5, ¿el puntero se movió del inicio del arreglo?
6. Escriba las instrucciones necesarias para visualizar en la pantalla cada una de las componentes del vector **frase**,
    a. Utilizando notación de punteros con el puntero **punt**.
    b. Mediante subíndices del puntero **punt**.
    c. Para los casos anteriores, ¿en qué posición quedó el puntero punt? ¿se ha movido de la componente 0 del arreglo frase?

AHORA SÍ: ¡A TRABAJAR!

CONSIGNA



Para los desplazamientos en los arreglos, use notación de punteros. Siempre que sea posible declare los arreglos de forma dinámica.

1. La secuencia creciente

Escriba un programa en C que permita al usuario ingresar un arreglo de números enteros de tamaño variable y luego encuentre y muestre la subsecuencia contigua más larga de elementos estrictamente crecientes, junto con su longitud y la suma de sus elementos.

- Use asignación dinámica de memoria para el manejo de los arreglos.
- Implemente un módulo para cargar el arreglo de entrada y otro para mostrar arreglos.
- Si hay más de una subsecuencia de igual longitud, informe la primera.

Por ejemplo, dado el arreglo:

3	1	4	6	9	2	5	7

Se obtiene el subarreglo:

1	4	6	9	4	20

(los elementos, luego la longitud y la suma: 1 + 4 + 6 + 9 = 20)

2. Cadena con snake_case

Escriba un programa en C que lea una oración y copie el contenido de la misma a un nuevo arreglo, reemplazando cada grupo de espacios consecutivos por un único guión bajo y convirtiendo todas las letras a minúscula, es decir:

- Todas las palabras quedan en minúscula.

- Las palabras se separan por un solo guión bajo (_).
- No debe quedar ningún guión bajo al principio ni al final de la cadena resultante, aunque la oración original tenga espacios sobrantes en esos lugares.

Use punteros para recorrer tanto la oración original como el arreglo de destino (no usar índices).

Implemente un módulo que reciba la oración y devuelva la cadena convertida, y otro para mostrar cadenas.

Finalmente, imprima la cadena resultante.

Nota: la oración ingresada no contendrá letras acentuadas ni caracteres especiales (ñ, ü, etc.), solo letras del alfabeto inglés, dígitos y espacios.

Por ejemplo, dada la oración: “ PRogramacion Es mi Materia FaVorita “

Salida: “programacion_es_mi_materia_favorita”

3. El juego del sapo



En una Kermés se presenta el clásico juego del sapo. Cada jugador tira fichas de metal hacia una mesa que tiene la “vieja” (una cara redonda con la boca abierta), un sapo y 8 agujeros. El costo de cada tiro es de $1000. Los premios según dónde cae la ficha son:

- La boca de la vieja (valor 10) tiene un premio de $5000.
- La boca del sapo (valor 9) tiene un premio de $3000.
- Cualquiera de los 8 agujeros (valores 1 a 8) tiene un premio de $1000.
- Si la ficha no acierta a ninguno (valor 0), no hay premio y se descuentan $500 adicionales.

Realice un programa que permita ingresar la cantidad de tiros que desea realizar el jugador.

Luego, simule los tiros generando cada puntaje de forma aleatoria entre 0 y 10 (`rand`). Por último, muestre el resumen de tiros, cuánta plata gastó o perdió, cuánta plata ganó y el saldo final del jugador.

- Use asignación dinámica de memoria para el arreglo de puntajes.
- Recorra los arreglos usando punteros (no usar índices).
- Utilice las siguientes funciones:

```
//Función para cargar los puntajes de los tiros.
void cargarTiros(int *puntos, int tiros);
//Función que calcula cuánto ganó el jugador.
float calcularGanancia(int *puntos, int tiros, float *perdida);
```



Sugerencia: modifique el programa anterior para que funcione con un número indeterminado de jugadores. Además de mostrar los resultados de cada jugador, al finalizar el juego para todos los usuarios muestre el monto total de gasto/pérdida, el monto total en ganancias y el saldo final de la kermés. Elija la marca final que considere apropiada (por ejemplo, ingresar 0 como cantidad de tiros).

Para esto modifique la función **calcularGanancia** con el siguiente prototipo:

```
float calcularGanancia(int *puntos, int tiros, float *perdida, float *totalPerdida, float *totalGanancia)
```

donde:

- **totalPerdida:** acumula el pago/descuento efectuado por los jugadores.
- **totalGanancia:** acumula la cantidad de premios ganados por los jugadores.

Por ejemplo, un jugador que realiza 5 tiros y obtiene los puntajes 10, 0, 4, 9 y 0:

- Gastó: 5 × $1000 = $5000, más 2 × $500 por los dos tiros que erraron, es decir $6000 de gasto/pérdida.
- Ganó: $5000 (vieja) + $1000 (agujero 4) + $3000 (sapo) = $9000.
- Saldo final: $9000 − $6000 = **$3000**.